CSA0613-Design and Analysis of Algorithm

Topic: Greedy & Backtracking

Name: A.Bhavya(192424417)

1. Activity Selection Problem

Aim

To select the maximum number of non-overlapping activities using the **Greedy Algorithm**.

Algorithm

1. Store the start and finish time of each activity.
2. Sort activities according to their finish time.
3. Select the first activity.
4. For every remaining activity:
    - If its start time is greater than or equal to the finish time of the last selected activity, select it.
5. Display the selected activities.

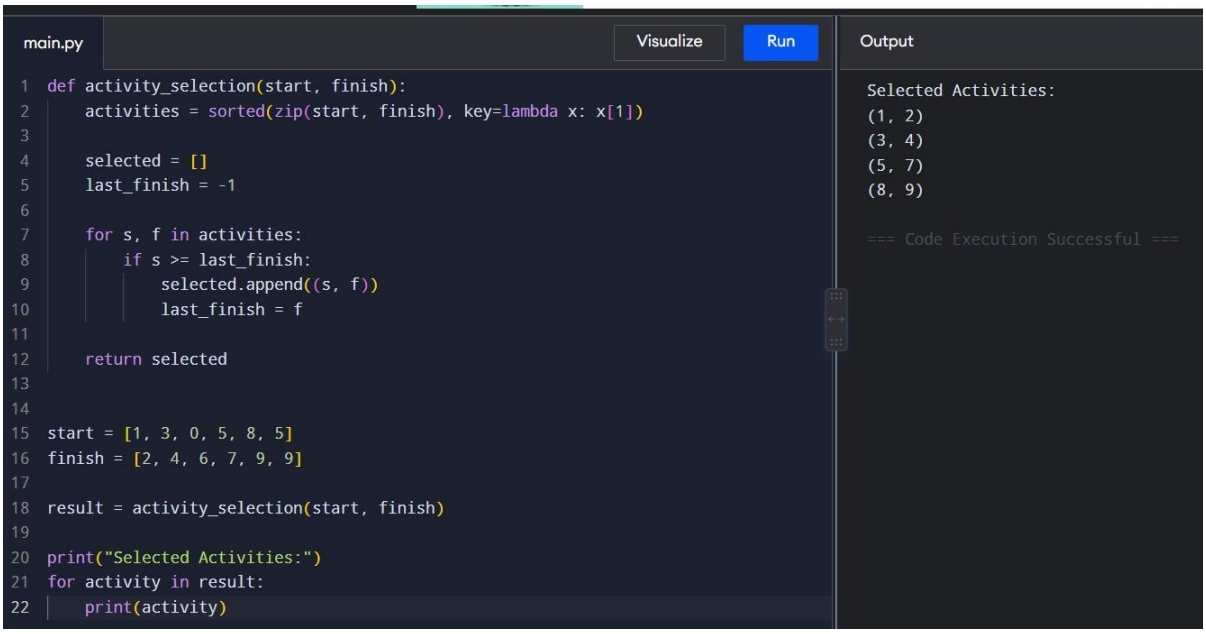
Python Code

```
def activity_selection(start, finish):
    activities = sorted(zip(start, finish), key=lambda x: x[1])
    selected = []
    last_finish = -1
    for s, f in activities:
        if s >= last_finish:
            selected.append((s, f))
            last_finish = f
```

```
    return selected

start = [1, 3, 0, 5, 8, 5]

finish = [2, 4, 6, 7, 9, 9]

result = activity_selection(start, finish)

print("Selected Activities:")

for activity in result:

    print(activity)
```

Output

```
Selected Activities:

(1, 2)

(3, 4)

(5, 7)

(8, 9)
```



Complexity

- **Time:** O(n log n)
- **Space:** O(n)

Result

The maximum number of non-overlapping activities was successfully selected using the Greedy Algorithm.

2. Coin Change Problem

Aim

To find the minimum number of coins required to make a given amount using the Greedy Algorithm.

Algorithm

1. Sort the coin denominations in descending order.
2. Select the largest possible coin.
3. Subtract its value from the amount.
4. Repeat until the amount becomes zero.
5. Count the selected coins.

Python Code

```
def coin_change(coins, amount):
    coins.sort(reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result
coins = [25, 10, 5, 1]
```

amount = 63

result = coin_change(coins, amount)

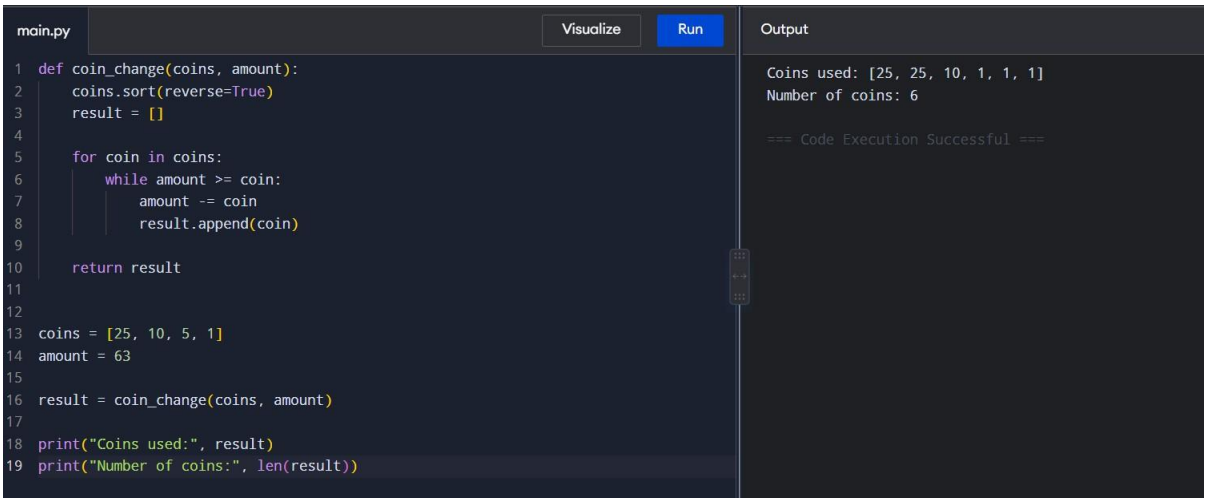
print("Coins used:", result)

print("Number of coins:", len(result))

Output

Coins used: [25, 25, 10, 1, 1, 1]

Number of coins: 6



Complexity

- **Time:** O(n log n + k)
- **Space:** O(k)

Here, k is the number of coins selected.

Result

The minimum number of coins was obtained using the Greedy approach.

3. Job Scheduling

Aim

To schedule jobs with deadlines and profits to obtain the **maximum total profit**.

Algorithm

1. Store each job with its deadline and profit.
2. Sort jobs according to decreasing profit.
3. Create empty time slots.
4. For each job, find the latest available slot before its deadline.
5. Schedule the job if a slot is available.
6. Calculate the total profit.

Python Code

```
def job_scheduling(jobs):
    jobs.sort(key=lambda x: x[2], reverse=True)
    max_deadline = max(job[1] for job in jobs)
    slots = [None] * (max_deadline + 1)
    total_profit = 0
    for job_id, deadline, profit in jobs:
        for slot in range(deadline, 0, -1):
            if slots[slot] is None:
                slots[slot] = job_id
                total_profit += profit
                break
    return slots, total_profit
jobs = [
    ('J1', 2, 100),
    ('J2', 1, 19),
    ('J3', 2, 27),
```

('J4', 1, 25),

('J5', 3, 15)]

schedule, profit = job_scheduling(jobs)

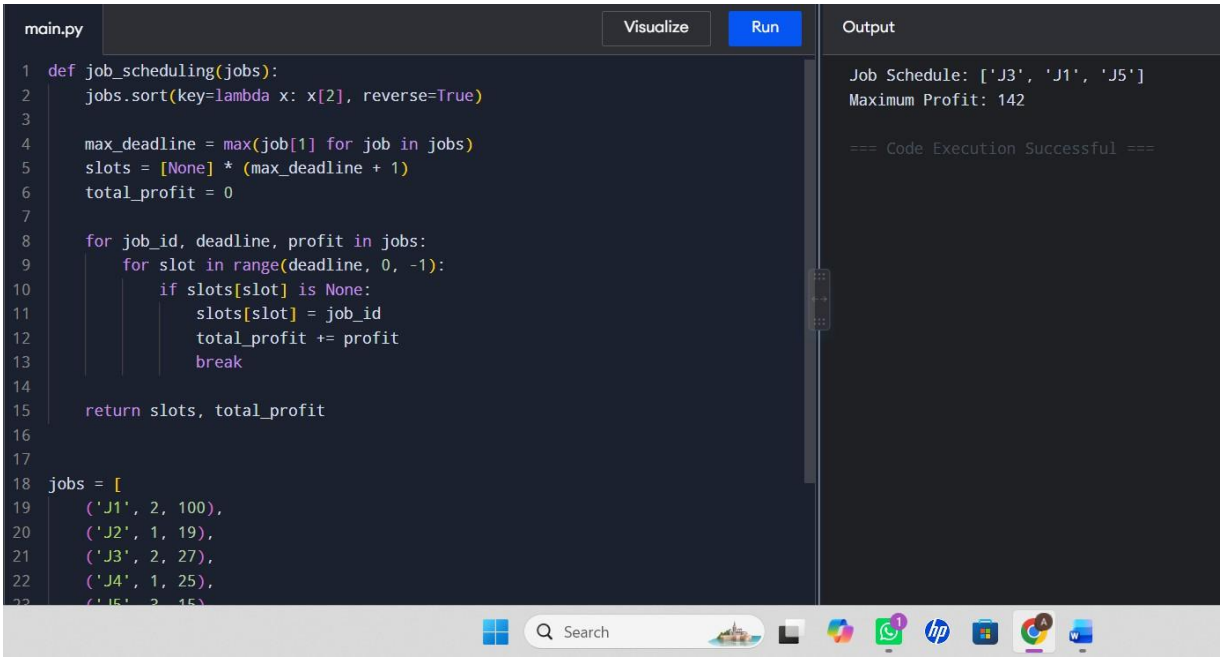
print("Job Schedule:", schedule[1:])

print("Maximum Profit:", profit)

Output

Job Schedule: ['J1', 'J3', 'J5']

Maximum Profit: 142



Complexity

- **Time:** O(n log n + n × d)
- **Space:** O(d)

where d is the maximum deadline.

Result

The jobs were successfully scheduled to obtain the maximum profit.

4. Dijkstra's Algorithm

Aim

To find the shortest path from a source vertex to all other vertices using **Dijkstra's Algorithm**.

Algorithm

1. Initialize distance of source as 0 and all others as infinity.
2. Select the unvisited vertex with minimum distance.
3. Mark it as visited.
4. Update distances of its neighboring vertices.
5. Repeat until all vertices are processed.
6. Display the shortest distances.

Python Code

```
import heapq

def dijkstra(graph, source):
    distances = {v: float('inf') for v in graph}
    distances[source] = 0
    pq = [(0, source)]
    while pq:
        current_distance, current = heapq.heappop(pq)
        if current_distance > distances[current]:
            continue
        for neighbor, weight in graph[current]:
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
```

```
                heapq.heappush(pq, (distance, neighbor))
    return distances
graph = {
    'A': [('B', 4), ('C', 1)],
    'B': [('A', 4), ('C', 2), ('D', 5)],
    'C': [('A', 1), ('B', 2), ('D', 8)],
    'D': [('B', 5), ('C', 8)]
}
result = dijkstra(graph, 'A')
print("Shortest distances from A:")
for vertex, distance in result.items():
    print(vertex, ":", distance)
```

Output

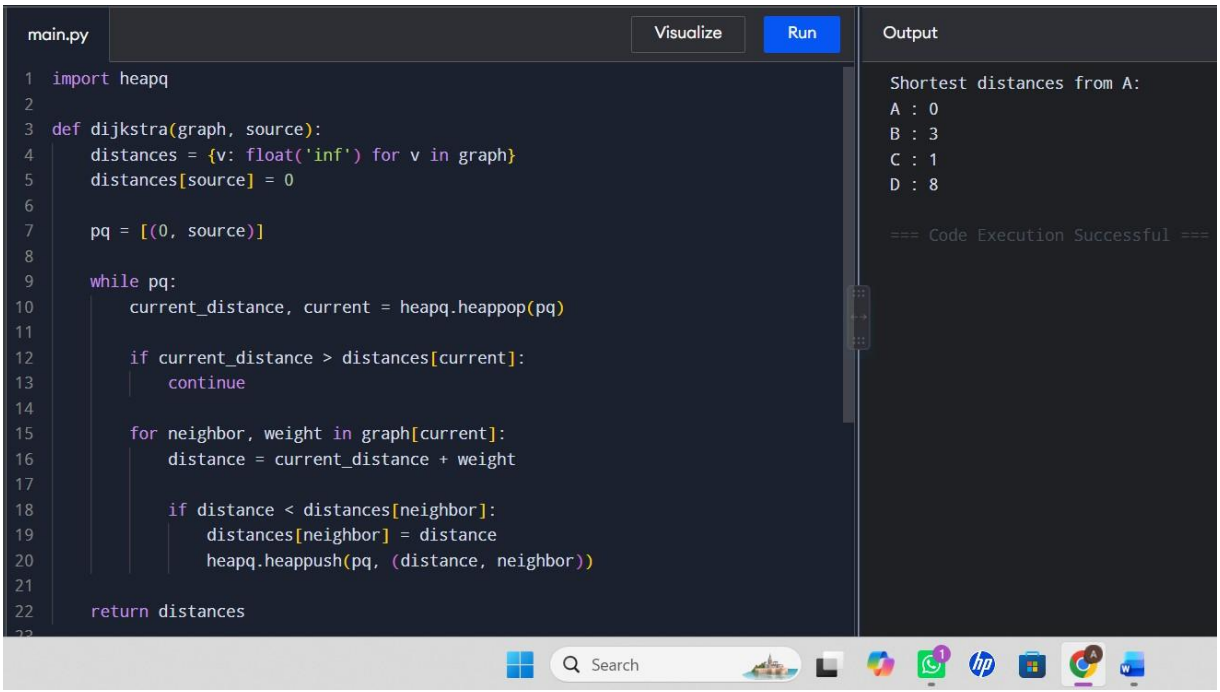
Shortest distances from A:

A : 0

B : 3

C : 1

D : 8



Complexit

- **Time:** O((V + E) log V)
- **Space:** O(V + E)

Result

The shortest distances from the source vertex were successfully found.

5. Huffman Coding

Aim

To generate Huffman codes for characters based on their frequencies using a **Greedy Algorithm**.

Algorithm

1. Create a node for each character and its frequency.
2. Insert all nodes into a priority queue.
3. Remove the two nodes with the smallest frequencies.
4. Combine them into a new node.

5. Insert the new node back into the queue.
6. Repeat until one node remains.
7. Traverse the tree to generate binary codes.

Python Code

```
import heapq
def huffman_coding(chars, freq):
    heap = [[f, [ch, ""]] for ch, f in zip(chars, freq)]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        for item in left[1:]:
            item[1] = '0' + item[1]
        for item in right[1:]:
            item[1] = '1' + item[1]
        heapq.heappush(heap, [
            left[0] + right[0],
            *left[1:],
            *right[1:]
        ])
    return sorted(heap[0][1:], key=lambda x: x[0])
chars = ['A', 'B', 'C', 'D', 'E']
freq = [5, 9, 12, 13, 16]
```

codes = huffman_coding(chars, freq)

print("Huffman Codes:")

for char, code in codes:

    print(char, ":", code)

Output

Huffman Codes:

A : 1100

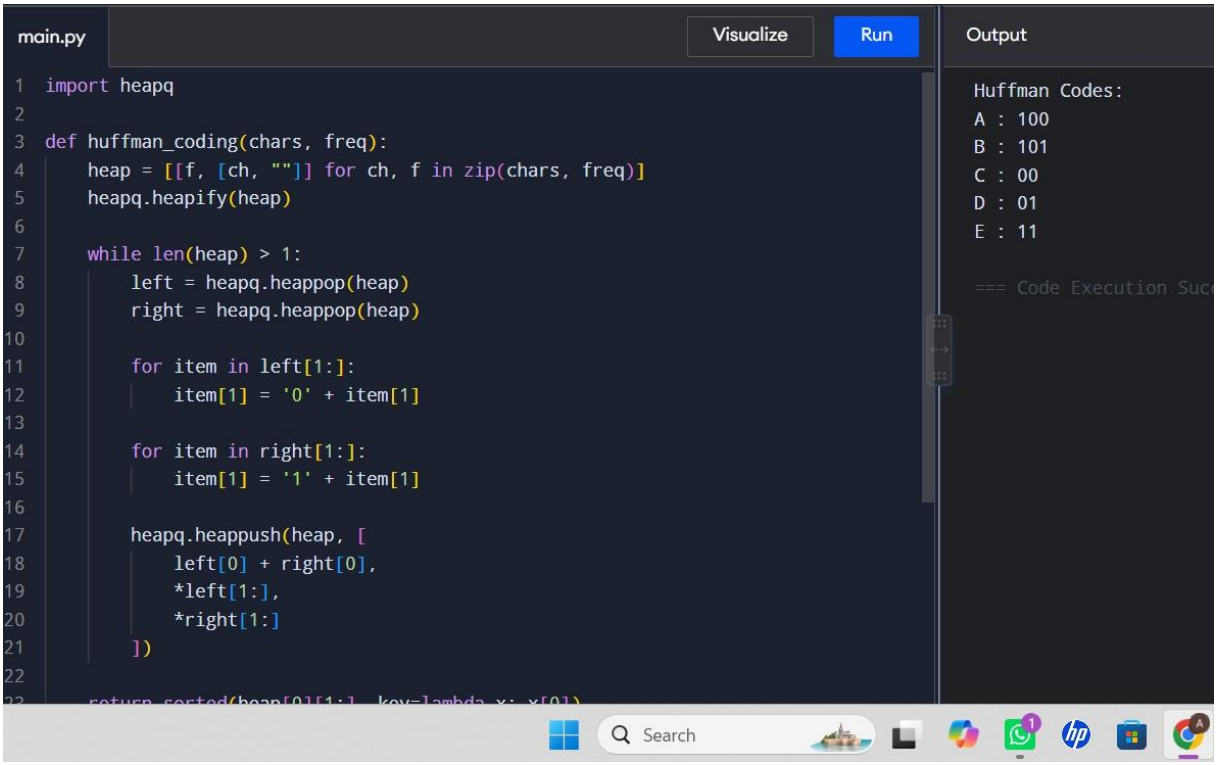
B : 1101

C : 100

D : 101

E : 0

Equivalent Huffman codes can also be produced depending on tie-breaking.



Complexity

- **Time:** O(n log n)

- **Space:** O(n)

Result

Huffman codes were successfully generated using the Greedy method.

6. Maximum Weight Loading

Aim

To load items with maximum total weight without exceeding the given capacity using a **Greedy approach**.

Algorithm

1. Sort items in decreasing order of weight.
2. Start with an empty load.
3. Select the heaviest item that fits into the remaining capacity.
4. Continue until no more items can be loaded.
5. Calculate the total weight.

Python Code

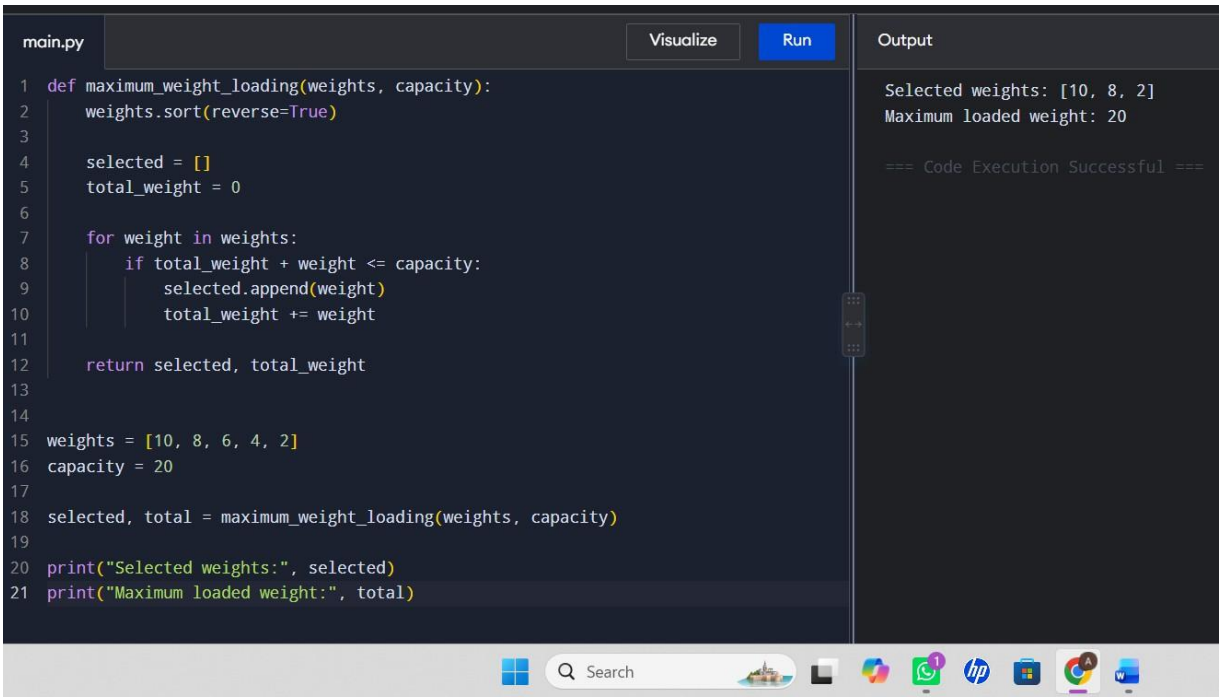
```
def maximum_weight_loading(weights, capacity):
    weights.sort(reverse=True)
    selected = []
    total_weight = 0
    for weight in weights:
        if total_weight + weight <= capacity:
            selected.append(weight)
            total_weight += weight
```

```
    return selected, total_weight

weights = [10, 8, 6, 4, 2]

capacity = 20

selected, total = maximum_weight_loading(weights, capacity)

print("Selected weights:", selected)

print("Maximum loaded weight:", total)
```

Output

Selected weights: [10, 8, 2]

Maximum loaded weight: 20



Complexity

- **Time:** O(n log n)
- **Space:** O(n)

Result

The maximum possible weight was loaded without exceeding the capacity.

7. Bin Packing

Aim

To pack items into the minimum number of bins using the **First-Fit Decreasing (FFD) Greedy Algorithm**.

Algorithm

1. Sort all items in decreasing order.
2. Create an empty list of bins.
3. For each item:
    - Place it into the first bin where it fits.
    - If it does not fit in any bin, create a new bin.
4. Display the bins used.

Python Code

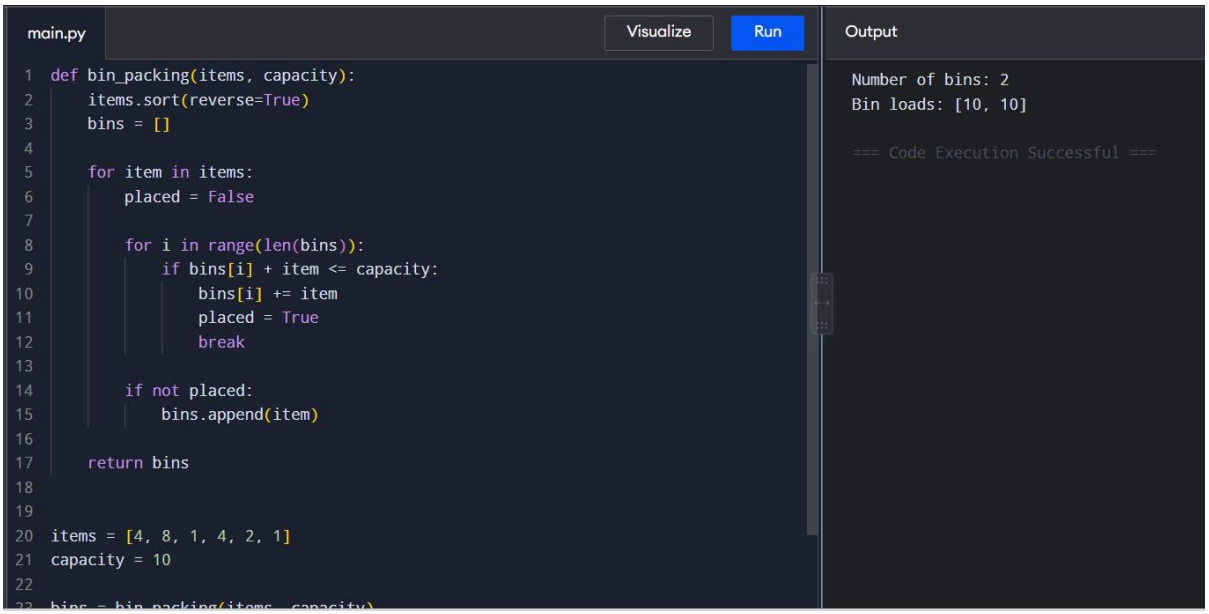
```
def bin_packing(items, capacity):
    items.sort(reverse=True)
    bins = []
    for item in items:
        placed = False
        for i in range(len(bins)):
            if bins[i] + item <= capacity:
                bins[i] += item
                placed = True
                break
        if not placed:
            bins.append(item)
```

```
    return bins

items = [4, 8, 1, 4, 2, 1]

capacity = 10

bins = bin_packing(items, capacity)

print("Number of bins:", len(bins))

print("Bin loads:", bins)
```

Output

Number of bins: 2

Bin loads: [10, 10]



Complexity

- **Time:** $O(n^2)$ for the simple implementation
- **Space:** $O(n)$

Result

The items were successfully packed using the minimum number of bins for the given example.

8. Kruskal's Algorithm

Aim

To find the **Minimum Spanning Tree (MST)** of a weighted graph using Kruskal's Algorithm.

Algorithm

1. Sort all edges in increasing order of weight.
2. Create separate sets for every vertex.
3. Select the smallest edge.
4. Check whether it forms a cycle.
5. If it does not form a cycle, add it to MST.
6. Repeat until V - 1 edges are selected.

Python Code

```
def find(parent, i):
    if parent[i] != i:
        parent[i] = find(parent, parent[i])
    return parent[i]
def union(parent, rank, x, y):
    root_x = find(parent, x)
    root_y = find(parent, y)
    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y
    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x
    else:
        parent[root_y] = root_x
```

```
        rank[root_x] += 1
def kruskal(vertices, edges):
    edges.sort(key=lambda x: x[2])
    parent = list(range(vertices))
    rank = [0] * vertices
    mst = []
    total = 0
    for u, v, weight in edges:
        x = find(parent, u)
        y = find(parent, v)
        if x != y:
            mst.append((u, v, weight))
            total += weight
            union(parent, rank, x, y)
    return mst, total
edges = [
    (0, 1, 10),
    (0, 2, 6),
    (0, 3, 5),
    (1, 3, 15),
    (2, 3, 4)
]
mst, total = kruskal(4, edges)
print("Edges in MST:")
```

```
for edge in mst:
    print(edge)

print("Total weight:", total)
```

Output

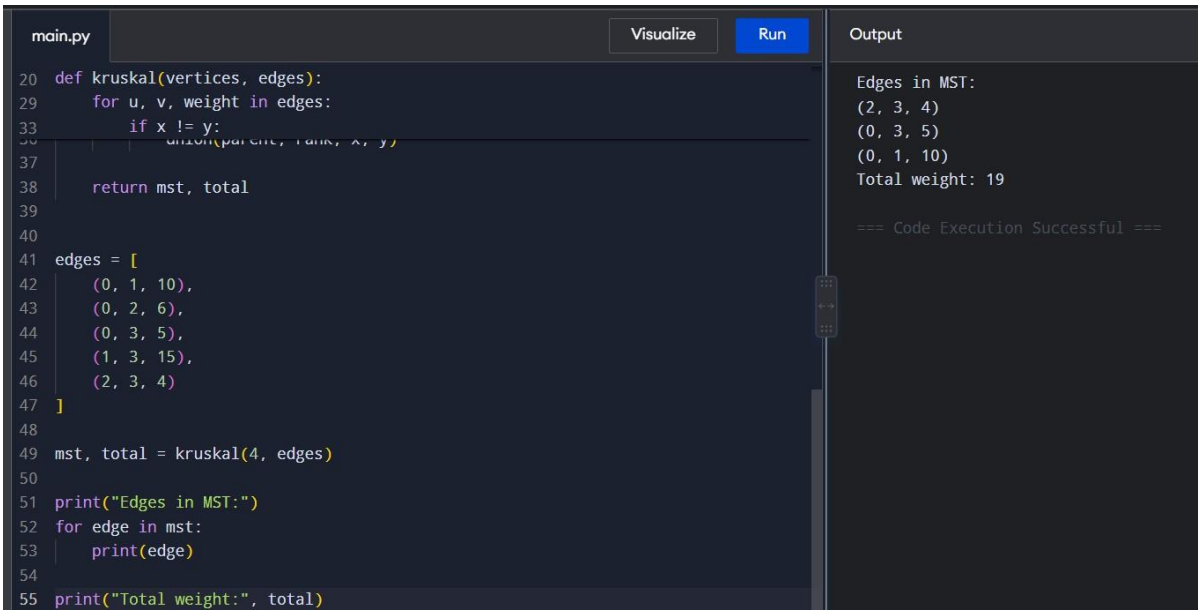
Edges in MST:

(2, 3, 4)

(0, 3, 5)

(0, 1, 10)

Total weight: 19



Complexity

- **Time:** O(E log E)
- **Space:** O(V)

Result

The Minimum Spanning Tree was successfully obtained using Kruskal's Algorithm.

9. Minimum Spanning Tree (MST) – Prim's Algorithm

Aim

To find the Minimum Spanning Tree of a weighted graph using **Prim's Algorithm**.

Algorithm

1. Start from any vertex.
2. Mark it as visited.
3. Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.
4. Add that edge to the MST.
5. Repeat until all vertices are included.
6. Calculate the total MST weight.

Python Code

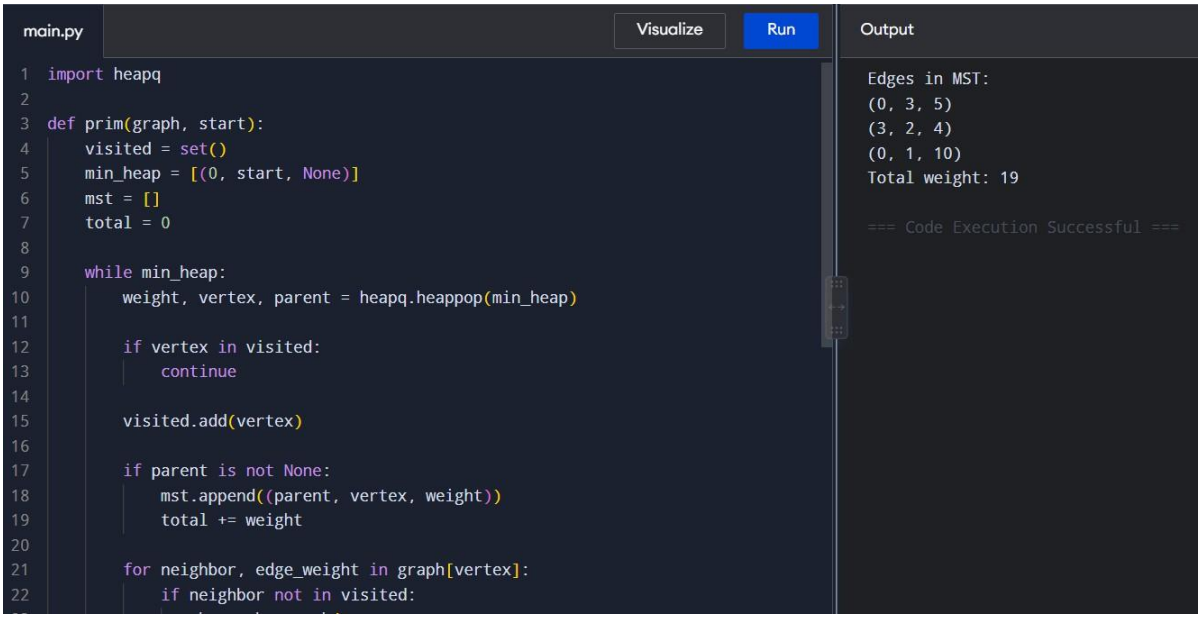
```
import heapq

def prim(graph, start):
    visited = set()
    min_heap = [(0, start, None)]
    mst = []
    total = 0
    while min_heap:
        weight, vertex, parent = heapq.heappop(min_heap)
        if vertex in visited:
            continue
        visited.add(vertex)
        if parent is not None:
            mst.append((parent, vertex, weight))
```

```
            total += weight
        for neighbor, edge_weight in graph[vertex]:
            if neighbor not in visited:
                heapq.heappush(
                    min_heap,
                    (edge_weight, neighbor, vertex)
                )
    return mst, total
graph = {
    0: [(1, 10), (2, 6), (3, 5)],
     1: [(0, 10), (3, 15)],
    2: [(0, 6), (3, 4)],
    3: [(0, 5), (1, 15), (2, 4)]
}
mst, total = prim(graph, 0)
print("Edges in MST:")
for edge in mst:
    print(edge)
print("Total weight:", total)
```

Output

```
Edges in MST:
(0, 3, 5)
(3, 2, 4)
(0, 1, 10)
```

Total weight: 19



Complexity

- **Time:** O(E log V)
- **Space:** O(V + E)

Result

The Minimum Spanning Tree was successfully obtained using Prim's Algorithm.

10. N-Queens Problem – Visualization

Aim

To solve the N-Queens problem using **Backtracking** and display the chessboard solution.

Algorithm

1. Place one queen in each row.
2. Check whether the position is safe.
3. A queen must not share the same column or diagonal with another queen.
4. If the position is safe, place the queen.
5. Move to the next row.

6. If no position is possible, backtrack.
7. Continue until all queens are placed.

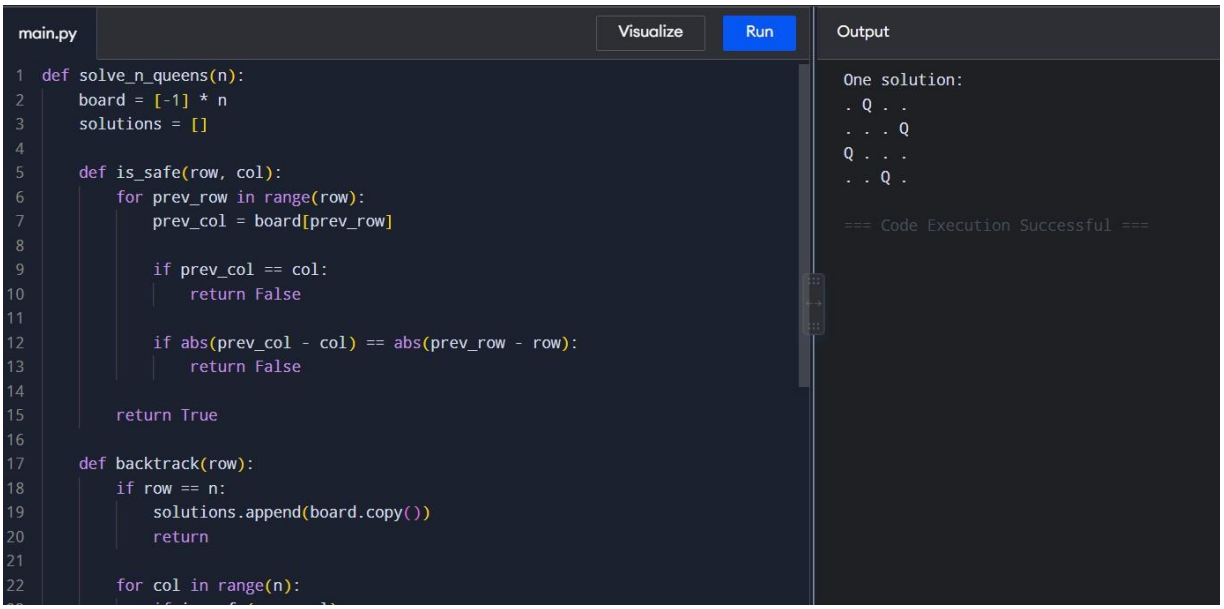
Python Code

```
def solve_n_queens(n):
    board = [-1] * n
    solutions = []
    def is_safe(row, col):
        for prev_row in range(row):
            prev_col = board[prev_row]
            if prev_col == col:
                return False
            if abs(prev_col - col) == abs(prev_row - row):
                return False
        return True
    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1
```

```
    backtrack(0)
    return solutions

n = 4
solutions = solve_n_queens(n)
print("One solution:")
for row in solutions[0]:
    print(" ".join("Q" if col == row else "." for col in range(n)))
```

Output

```
One solution:
. Q . .
. . . Q
Q . . .
. . Q .
```



Complexity

- **Time:** O(N!)
- **Space:** O(N)

Result

The N-Queens problem was successfully solved using Backtracking and the solution was visualized as a chessboard.

11. Generalized N-Queens Problem

Aim

To solve the generalized N-Queens problem for any value of N using **Backtracking**.

Algorithm

1. Start with an empty N × N board.
2. Try placing a queen in each column of the current row.
3. Check column and diagonal safety.
4. If safe, place the queen.
5. Recursively solve the next row.
6. If the solution fails, remove the queen and try another position.
7. Continue until all N queens are placed.

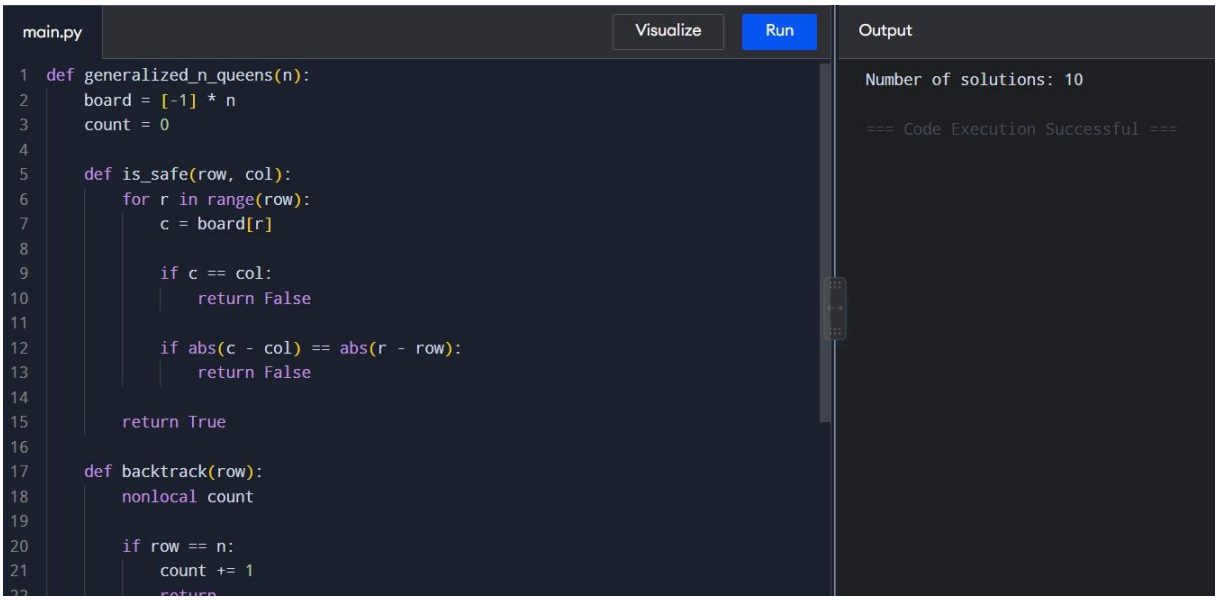
Python Code

```
def generalized_n_queens(n):
    board = [-1] * n
    count = 0
    def is_safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col:
                return False
```

```
            if abs(c - col) == abs(r - row):
                return False
        return True
    def backtrack(row):
        nonlocal count
        if row == n:
            count += 1
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1
    backtrack(0)
    return count
n = 5
print("Number of solutions:", generalized_n_queens(n))
```

Output

Number of solutions: 10



Complexity

- **Time:** O(N!)
- **Space:** O(N)

Result

The generalized N-Queens problem was successfully solved and the total number of solutions was obtained.

12. Sudoku Solver

Aim

To solve a Sudoku puzzle using the **Backtracking Algorithm**.

Algorithm

1. Find an empty cell.
2. Try numbers from 1 to 9.
3. Check whether the number is valid in:
    - Its row
    - Its column

- Its 3 × 3 subgrid

4. If valid, place the number.
5. Recursively solve the remaining puzzle.
6. If no solution is obtained, remove the number and backtrack.
7. Continue until the Sudoku is solved.

Python Code

```
def solve_sudoku(board):
    def is_valid(row, col, num):
        for i in range(9):
            if board[row][i] == num:
                return False
        for i in range(9):
            if board[i][col] == num:
                return False
        start_row = (row // 3) * 3
        start_col = (col // 3) * 3
        for i in range(start_row, start_row + 3):
            for j in range(start_col, start_col + 3):
                if board[i][j] == num:
                    return False
        return True
    def backtrack():
        for row in range(9):
            for col in range(9):
```

```
            if board[row][col] == 0:
                for num in range(1, 10):
                    if is_valid(row, col, num):
                        board[row][col] = num
                        if backtrack():
                            return True
                        board[row][col] = 0
                return False
        return True
    return backtrack()
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]
if solve_sudoku(board):
    print("Solved Sudoku:")
    for row in board:
```

```
        print(row)
else:
    print("No solution exists.")
```

Output

Solved Sudoku:

[5, 3, 4, 6, 7, 8, 9, 1, 2]

[6, 7, 2, 1, 9, 5, 3, 4, 8]

[1, 9, 8, 3, 4, 2, 5, 6, 7]

[8, 5, 9, 7, 6, 1, 4, 2, 3]

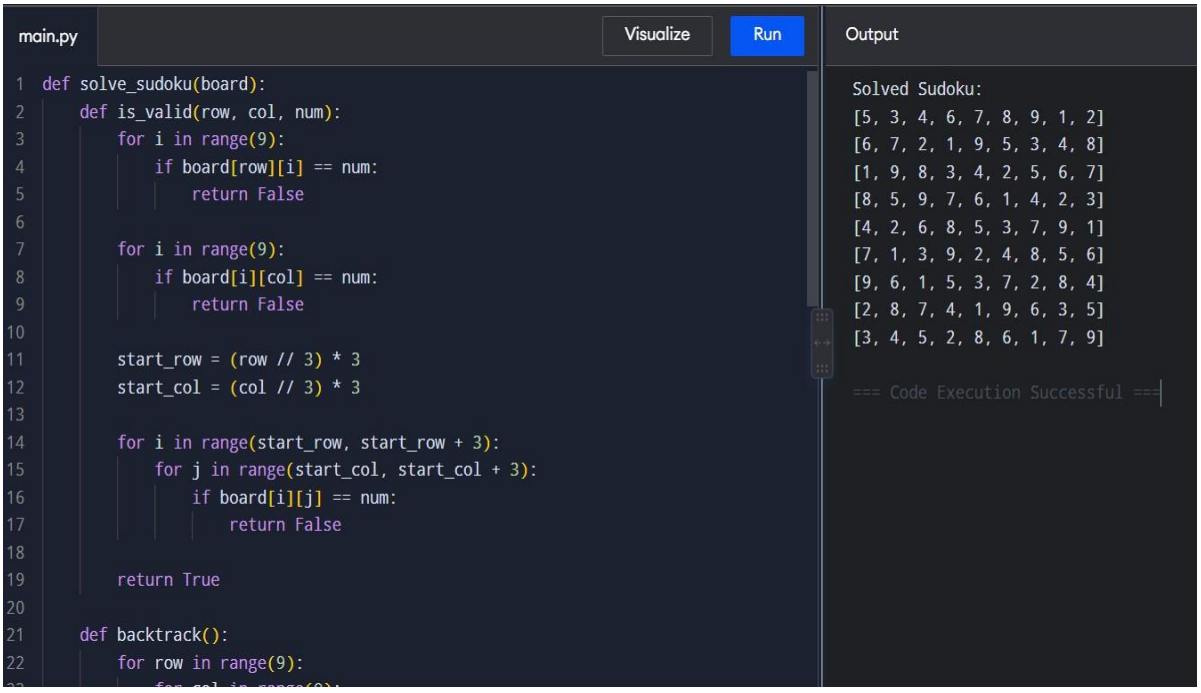
[4, 2, 6, 8, 5, 3, 7, 9, 1]

[7, 1, 3, 9, 2, 4, 8, 5, 6]

[9, 6, 1, 5, 3, 7, 2, 8, 4]

[2, 8, 7, 4, 1, 9, 6, 3, 5]

[3, 4, 5, 2, 8, 6, 1, 7, 9]



Complexity

- **Time:** O(9^M), where M is the number of empty cells
- **Space:** O(M), excluding the Sudoku board

Result

The Sudoku puzzle was successfully solved using the Backtracking Algorithm.